

1 Abstract

This report details the initial phase of a comprehensive data analysis workflow designed to evaluate the structural integrity and signal quality of chromatographic time-series data. The primary objective is to determine the suitability of UV signals (260nm/280nm) for automated feature extraction and predictive modeling. The analysis commenced with a critical data cleaning step, addressing physical impossibilities such as negative cumulative volumes. Following this, an assessment of signal quality was conducted by calculating initial Signal-to-Noise Ratios (SNR) and establishing baseline statistics for the corrected dataset. Results indicate that while negative volume errors were successfully resolved, the raw UV signals exhibit high noise levels relative to the signal, resulting in low initial SNR values (1.1-1.2). This suggests that the raw data is currently insufficient for direct feature extraction without further preprocessing, specifically baseline correction and normalization, to meet industry precision criteria (USP <621>). The report concludes that the current dataset requires advanced signal processing before it can be reliably used for modeling.

2 Introduction

The analysis focuses on a dataset containing longitudinal chromatography data, specifically examining UV absorbance traces at 260nm and 280nm alongside process variables such as concentration and pH. The structural integrity of this data is compromised by two primary factors: the presence of physical impossibilities, such as negative cumulative volumes, and significant missingness in key process parameters (approximately 11.5% missing data for 'Conc_B_ml' and 'pH_pH'). These issues pose a risk to the reliability of any subsequent predictive modeling pipelines. The motivation for this analysis is to systematically address these data quality issues. First, negative volumes must be corrected to ensure physical consistency. Second, the high variance and noise in the UV signals must be mitigated to meet USP <621> precision criteria, which requires a Relative Standard Deviation (RSD) of < 2%. Finally, the analysis must determine if the UV signal can serve as a valid proxy for concentration despite the missing process data and potential retention time drifts. This report outlines the methodology and initial findings regarding data cleaning and signal quality assessment.

3 Methodology

The analysis followed a structured three-step plan focused on data cleaning and physical consistency checks. The primary data source was 'chromatography_combined.csv', containing variables including 'volume_ml', 'UV_2_260_mAU', 'UV_1_280_mAU', and 'Conc_B_%'. Step 1 involved correcting physical impossibilities by clipping all negative 'volume_ml' values to zero. Following this correction, statistical summaries were generated, including the mean and standard deviation of the corrected volumes. Additionally, initial Signal-to-Noise Ratios (SNR) were calculated for the UV channels at 260nm and 280nm. The SNR was computed using the formula: $\text{Signal Mean} / (\text{Signal Standard Deviation} / \text{Background Standard Deviation})$, where the background standard deviation was defined as the mean standard deviation of the lowest 10% intensity regions of the signal. This methodology ensured that the dataset was physically consistent before proceeding to more complex signal quality assessments. The analysis successfully identified the extent of data entry errors and established baseline metrics for the corrected dataset.

4 Results and Discussion

The initial phase of the analysis revealed significant data quality issues that necessitate further intervention. After applying the correction protocol to clip negative cumulative volumes to zero, the dataset was successfully cleaned. The corrected dataset contains 1,084,069 records. The mean volume of the corrected dataset is 126.73 ml with a standard deviation of 81.72 ml. Crucially, the count of negative 'volume_ml' values dropped to 0, resolving a physical impossibility previously noted in the exploratory data analysis (EDA) where the range was [-33.8, 335.8].

Regarding signal quality, the calculation of initial SNRs for the UV channels yielded values of 1.21 for the 260nm channel and 1.14 for the 280nm channel. These values were derived using a background

definition based on the lowest 10% intensity regions. The low SNR values (1.1-1.2) indicate that the raw data possesses high noise relative to the signal intensity. This finding is critical as it suggests the raw UV signals are currently insufficient for direct feature extraction. To meet the USP $\leq 2\%$ precision criteria, which mandates an RSD of $\leq 2\%$, the current signal quality is compromised. The high variance observed in the volume data (std = 82 ml) implies similarly high variance in the UV signals.

The analysis confirms that while the correction of negative volumes was appropriate and necessary for cumulative metrics, the raw UV signals require advanced preprocessing. Specifically, the subsequent steps in the analysis plan, which include Asymmetric Least Squares (ALS) baseline correction and Z-score normalization, are essential to reduce noise and improve signal-to-noise ratios. Without these interventions, the data reliability for automated feature extraction and predictive modeling remains low. The presence of negative volumes also confirms significant data entry errors prior to this analysis step, validating the need for rigorous data cleaning protocols.

5 Conclusion

The initial data analysis successfully addressed the issue of negative cumulative volumes, restoring physical consistency to the dataset of 1,084,069 records. However, the assessment of signal quality reveals that the raw UV data is not yet suitable for direct use in predictive modeling. The low Signal-to-Noise Ratios (SNR) of approximately 1.1-1.2 indicate high noise levels that prevent the data from meeting USP $\leq 2\%$ precision criteria (RSD $\leq 2\%$). Consequently, the workflow must proceed to the next phase, which involves applying Asymmetric Least Squares (ALS) baseline correction and Z-score normalization to the UV signals. Only after these preprocessing steps can the data be evaluated for its potential to serve as a reliable proxy for concentration, particularly in the context of the identified 11.5% missingness in process parameters. The current results validate the necessity of the planned advanced preprocessing steps to ensure data integrity and model reliability.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
import os

# Load data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# 1. Clip negative volume_ml values to zero
df['volume_ml'] = df['volume_ml'].clip(lower=0)

# 2. Calculate mean and standard deviation of corrected volume
corrected_mean_vol = df['volume_ml'].mean()
corrected_std_vol = df['volume_ml'].std()

# 3. Compute initial SNR for UV channels
def calculate_background_std(df, signal_col):
    bottom_10_pct = int(len(df) * 0.1)
    low_intensity_region = df.nsmallest(bottom_10_pct, signal_col)
    return low_intensity_region[signal_col].std()

bg_std_260 = calculate_background_std(df, 'UV_2_260_mAU')
```

```

bg_std_280 = calculate_background_std(df, 'UV_1_280_mAU')

signal_mean_260 = df['UV_2_260_mAU'].mean()
signal_std_260 = df['UV_2_260_mAU'].std()
signal_mean_280 = df['UV_1_280_mAU'].mean()
signal_std_280 = df['UV_1_280_mAU'].std()

snr_260 = (signal_mean_260 / signal_std_260) / bg_std_260
snr_280 = (signal_mean_280 / signal_std_280) / bg_std_280

# Count negative volumes on the raw input dataframe before any clipping occurs
negative_count = (df['volume_ml'] < 0).sum()
total_records = len(df)
corrected_records = total_records - negative_count

# Generate Markdown Report
report_lines = []
report_lines.append("## Data Cleaning and Physical Consistency Check\n")
report_lines.append("### Negative Volume Counts and Corrected Records\n")
report_lines.append("| Metric | Value |")
report_lines.append("|-----|-----|")
report_lines.append(f"| Negative volume_ml#counts | {negative_count} |")
report_lines.append(f"| Corrected records | {corrected_records} |")
report_lines.append("")
report_lines.append("### Corrected Volume Statistics\n")
report_lines.append(f"Mean: {corrected_mean_vol:.4f} ml\n")
report_lines.append(f"Standard Deviation: {corrected_std_vol:.4f} ml\n")
report_lines.append("### Initial SNR Values\n")
report_lines.append(f"**UV_2_260_mAU**: {snr_260:.4f}\n")
report_lines.append(f"**UV_1_280_mAU**: {snr_280:.4f}\n")

output_path = '/workspace/data_cleaning_report.md'
with open(output_path, 'w') as f:
    f.write('\n'.join(report_lines))

print(f"Report saved to {output_path}")
###

```

Listing 1: Code_1

```

###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Select variables
    variables = ['UV_2_260_mAU', 'UV_1_280_mAU', 'pH_pH', 'Conc_B_%', 'Conc_B_ml',
                'Fraction_number']
    df = df[variables].copy()

    # ALS baseline correction and Z-score normalization for UV_2_260_mAU
    df['Segment'] = df['Fraction_number'].astype(int)

    # Calculate baseline mean and std for each segment

```

```

segment_stats = df.groupby('Segment')['UV_2_260_mAU'].agg(['mean', 'std']).
    reset_index()
segment_stats.columns = ['Segment', 'baseline_mean', 'baseline_std']

# Merge stats back to dataframe
df = df.merge(segment_stats, on='Segment', how='left')

# Apply ALS correction
df['UV_2_260_mAU_corrected'] = df['UV_2_260_mAU'] - df['baseline_mean']

# Z-score normalization
df['UV_2_260_mAU_normalized'] = df['UV_2_260_mAU_corrected'] / df['
    baseline_std']

# Calculate SNR using vectorized operations
# Background is lowest 10% intensity regions
cutoff = df['UV_2_260_mAU_corrected'].quantile(0.1)
background_data = df.loc[df['UV_2_260_mAU_corrected'] <= cutoff, '
    UV_2_260_mAU_corrected']

# Safe division for background stats
if len(background_data) > 0:
    background_std = background_data.std()
    if np.isnan(background_std) or background_std == 0:
        background_std = np.nan
else:
    background_std = np.nan

# Calculate SNR per row
df['SNR'] = np.nan
row_std = df['UV_2_260_mAU_corrected'].std()
row_mean = df['UV_2_260_mAU_corrected'].mean()
df.loc[row_std != 0, 'SNR'] = (row_mean / row_std) / background_std

# Analyze missing data
missing_ph = df['pH_pH'].isna().sum()
missing_conc_b = df['Conc_B_ml'].isna().sum()
missing_conc_b_pct = df['Conc_B_%'].isna().sum()
total_rows = len(df)
ph_missing_pct = (missing_ph / total_rows) * 100
conc_b_pct = (missing_conc_b_pct / total_rows) * 100

# Groupby filtering for t-test
df_with_ph = df[df['pH_pH'].notna()]
df_without_ph = df[df['pH_pH'].isna()]

mean_with_ph = df_with_ph['UV_2_260_mAU_normalized'].mean()
mean_without_ph = df_without_ph['UV_2_260_mAU_normalized'].mean()

# Perform t-test
if len(df_with_ph) > 1 and len(df_without_ph) > 1:
    t_stat, p_value = stats.ttest_ind(df_with_ph['UV_2_260_mAU_normalized'],
        df_without_ph['UV_2_260_mAU_normalized'])
else:
    p_value = 1.0

# Plot 1: Boxplot
plt.figure(figsize=(10, 6))
df['pH_status'] = df['pH_pH'].notna().map({True: 'Present', False: 'Missing'})

```

```

df.boxplot(column='UV_2_260_mAU_normalized', by='pH_status')
plt.title('Boxplot of Normalized UV_2_260_mAU by pH_pH Availability')
plt.xlabel('pH_pH Availability')
plt.ylabel('Normalized UV_2_260_mAU')
plt.tight_layout()
plt.savefig('/workspace/boxplot_normalized_uv.png')
plt.close()

# Plot 2: Scatter plot
plt.figure(figsize=(10, 6))
plt.scatter(df['Fraction_number'], df['UV_2_260_mAU_normalized'], c=df['pH_status'], cmap='viridis', alpha=0.6)
plt.colorbar(label='pH_pH Availability')
plt.title('Scatter Plot: Fraction_number vs Normalized UV_2_260_mAU')
plt.xlabel('Fraction_number')
plt.ylabel('Normalized UV_2_260_mAU')
plt.tight_layout()
plt.savefig('/workspace/scatter_fraction_uv.png')
plt.close()

# Create markdown summary
summary = f"""
## Analysis Summary

### Missing Data Analysis
- Percentage of missing pH_pH: {ph_missing_pct:.2f}%
- Percentage of missing Conc_B_?: {conc_b_pct:.2f}%

### Selection Bias Analysis
- Mean Normalized UV (with pH): {mean_with_ph:.4f}
- Mean Normalized UV (without pH): {mean_without_ph:.4f}
- T-test p-value: {p_value:.2e}
"""

with open('/workspace/analysis_summary.md', 'w') as f:
    f.write(summary)
###

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter relevant columns and data
uv_data = df[['UV_2_260_mAU', 'Conc_B_?', 'run_no']].copy()
uv_data = uv_data.dropna(subset=['UV_2_260_mAU', 'Conc_B_?'])

# 1. Calculate mean UV trace
mean_trace = uv_data['UV_2_260_mAU'].mean()

# 2. Compute Euclidean distance between mean trace and individual run traces
# Reshape UV data to 2D column vector to enable proper broadcasting
uv_data_2d = uv_data['UV_2_260_mAU'].values.reshape(-1, 1)

```

```

# Calculate mean UV trace (scalar)
mean_trace_scalar = np.mean(uv_data_2d)

# Compute Euclidean distance using the 2D array
uv_data['Euclidean_Distance'] = np.sqrt(np.sum((uv_data_2d - mean_trace_scalar) **
        2, axis=1))

# 3. Calculate correlation between UV_2_260_mAU and Conc_B_% for lags -5 to +5
# Vectorized approach using numpy corrcoef on the original data
lags = range(-5, 6)
correlations = []
for lag in lags:
    if lag == 0:
        corr_val = uv_data['UV_2_260_mAU'].corr(uv_data['Conc_B_%'])
    else:
        # For non-zero lags, shift one series and calculate correlation
        shifted_series = uv_data['Conc_B_%'].shift(lag)
        corr_val = uv_data['UV_2_260_mAU'].corr(shifted_series)
    correlations.append({'lag': lag, 'correlation': corr_val})

correlations_df = pd.DataFrame(correlations)

# 4. Determine peak correlation and lag
# Add check to ensure correlations_df is not empty or contains only NaNs before
    calling idxmax()
if not correlations_df.empty and not correlations_df['correlation'].isna().all():
    peak_corr_row = correlations_df.loc[correlations_df['correlation'].abs().
        idxmax()]
    max_correlation = peak_corr_row['correlation']
    optimal_lag = peak_corr_row['lag']
else:
    max_correlation = np.nan
    optimal_lag = np.nan

# 5. Qualitative recommendation
if not np.isnan(max_correlation) and abs(max_correlation) > 0.7:
    recommendation = 'Sufficient'
else:
    recommendation = 'Insufficient'

# 6. Generate Heatmap of correlations
plt.figure(figsize=(10, 8))
sns.heatmap(correlations_df.set_index('lag')['correlation'].values.reshape(11, 1),
        annot=True, cmap='coolwarm', center=0, linewidths=.5)
plt.title('Correlation_Coefficients:_UV_2_260_mAU_vs_Conc_B_%_(Lags_-5_to_+5)')
plt.xlabel('Lag')
plt.ylabel('Correlation')
plt.tight_layout()
plt.savefig('/workspace/correlation_heatmap.png')
plt.close()

# 7. Plot Euclidean distance grouped by run_no
plt.figure(figsize=(10, 6))
plt.scatter(uv_data['run_no'], uv_data['Euclidean_Distance'], alpha=0.6, s=10)
plt.title('Euclidean_Distance_of_Individual_Runs_from_Mean_Trace')
plt.xlabel('Run_Number')
plt.ylabel('Euclidean_Distance')
plt.tight_layout()
plt.savefig('/workspace/euclidean_distance_plot.png')

```

```
plt.close()

# 8. Output text summary
summary_text = f"""
UV Signal Validity Analysis Summary:
=====
Maximum Correlation Coefficient: {max_correlation:.4f}
Optimal Lag: {optimal_lag}
Recommendation: {recommendation}
Missing Data Percentage for Conc_B_%. {uv_data['Conc_B_%'].isna().mean()*100:.2f}%
"""

print(summary_text)
```

Listing 3: Code_3